re3

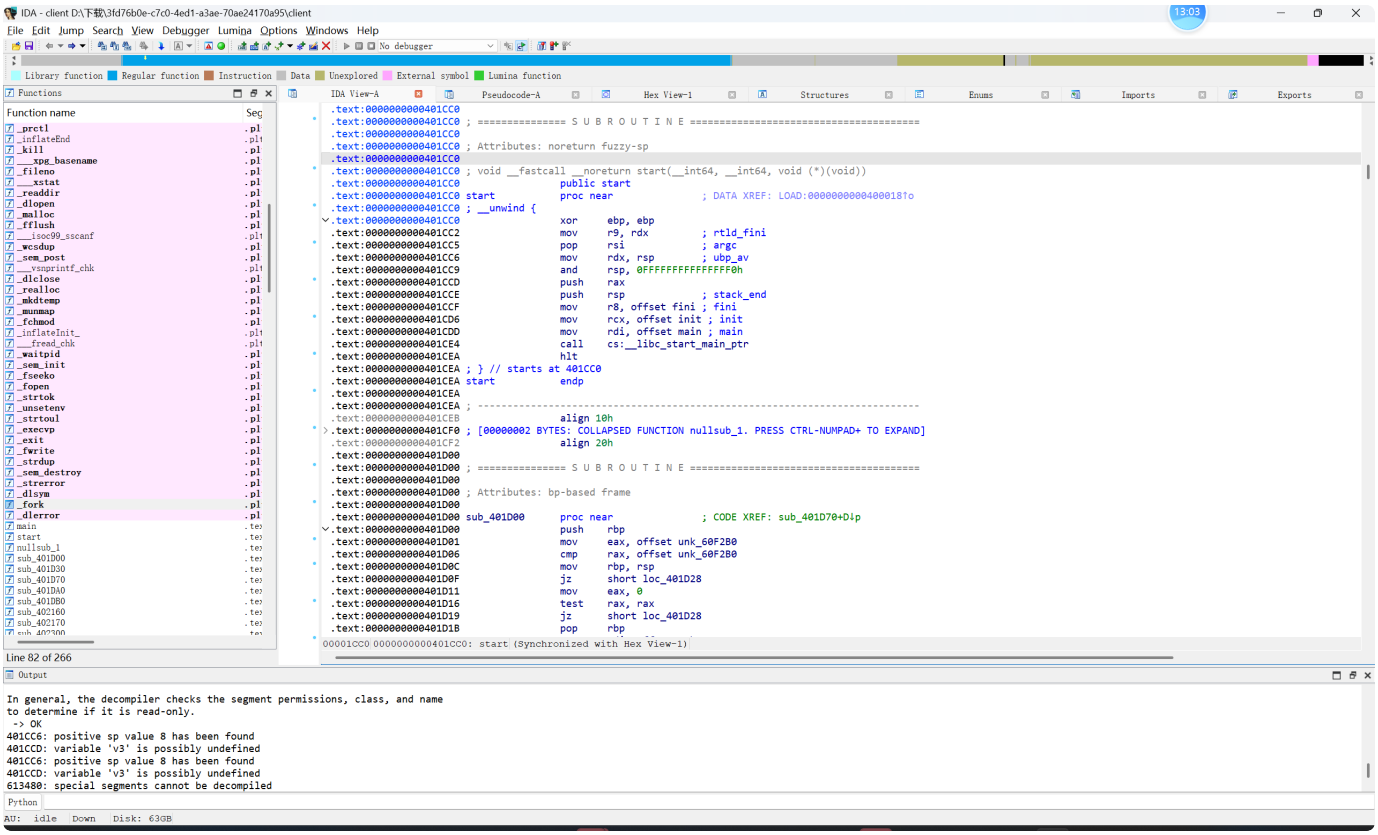
题目给了两个附件

- `client`：木马客户端程序
- `capture.pcap`：抓到的网络流量

题目提示：结合逆向和流量分析，确定木马泄露了什么敏感数据，并解出 flag。

我们先静态分析client，

用 IDA Pro MCP 连接当前打开的 `client`



client是一个 PyInstaller 打包的 Linux ELF。

真实业务逻辑不在 bootloader，而在内嵌 Python 字节码和扩展模块中。

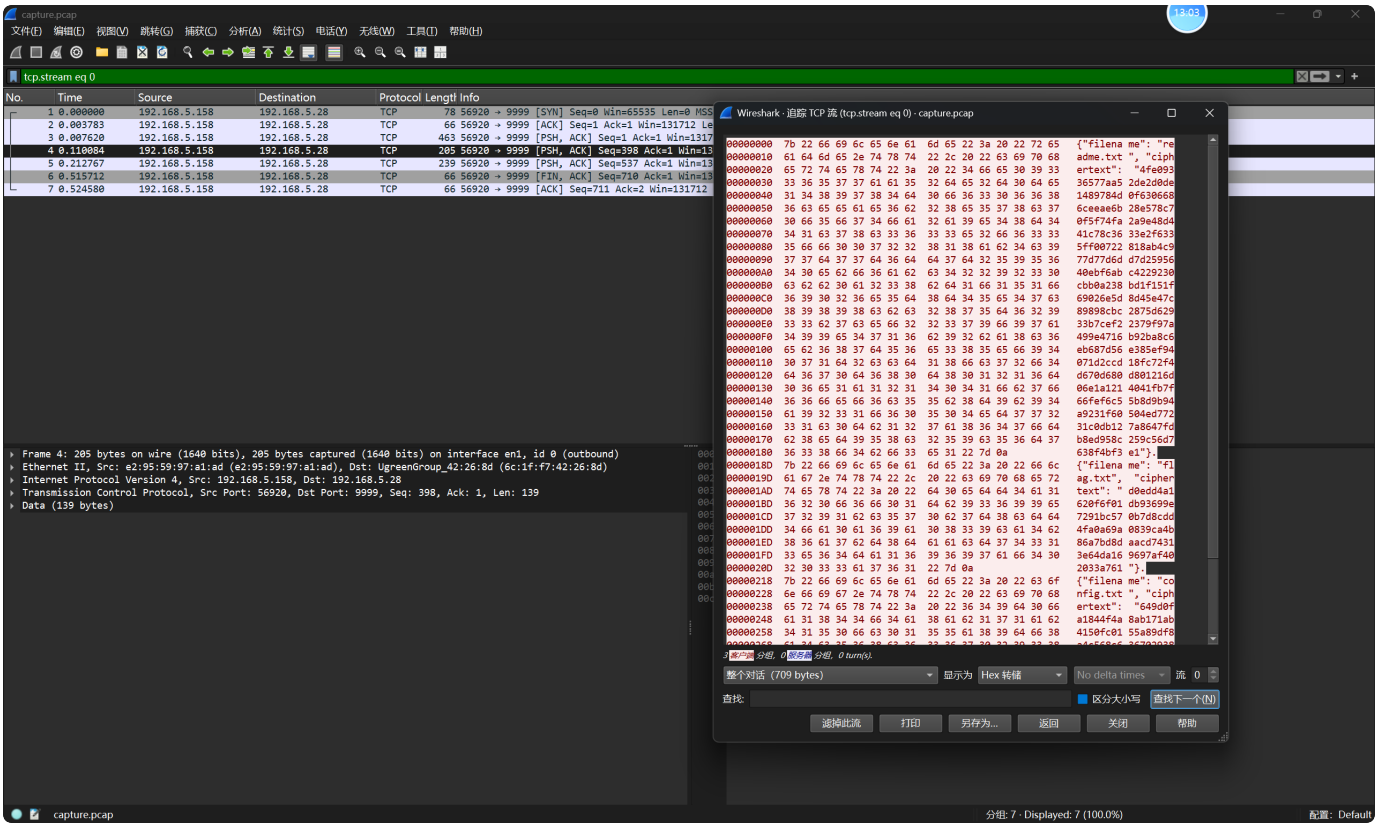
随后查看 PyInstaller 归档内容，关键文件有：

- 主脚本字节码：`client`（pyc）
- 加密扩展：`crypt_core.so`

接着我们要还原python主逻辑

提取并还原主脚本后，得到核心行为：

1. 程序通过 TCP 连接到服务端 `SERVER_PORT = 9999`
2. 会读取并外传 3 个文件（经 `_oe` 解混淆后）：
   - `readme.txt`
   - `flag.txt`
   - `config.txt`
3. 每个文件被加密后，以 JSON 行发送，字段为：
   - `filename`
   - `ciphertext`
4. 加密调用是：
   - `crypt_core.encode_data(plaintext, key[:16])`



使用 Wireshark 打开 capture.pcap，追踪 TCP 流（端口 9999），提取到三个 JSON 行，对应三个文件的密文（十六进制形式）：

readme.txt 的密文（较长）

flag.txt 的密文：
d0edd4a1620f6f01db93699e7291bc570b7d8cdd4fa0a69a0839ca4b86a7bd8daacd74313e64da169697af402033a761

config.txt 的密文

从 pcap 的 TCP payload 提取到 3 条外传 JSON：

- `readme.txt` 对应一段 hex 密文
- `flag.txt` 对应一段 hex 密文
- `config.txt` 对应一段 hex 密文

说明木马确实按代码逻辑完成了三次文件泄露。

我们有了完整的十六进制转储，包含了三个JSON行的数据。我们已经从之前的分析中知道这三个文件分别是readme.txt, flag.txt, config.txt。现在需要解密flag.txt。

```
┌──(kali㉿kali)-[~/Desktop/pyinstxtractor]
└─$ cd client_extracted
ls -la *.so
-rw-rw-r-- 1 kali kali 53344 Mar 14 06:17 crypt_core.so

┌──(kali㉿kali)-[~/Desktop/pyinstxtractor/client_extracted]
└─$ objdump -s -j .rodata crypt_core.so

crypt_core.so:     file format elf64-x86-64

Contents of section .rodata:
 a000 53686172 65642043 7974686f 6e207479  Shared Cython ty
 a010 70652025 2e323030 73206973 206e6f74  pe %.200s is not
 a020 20612074 79706520 6f626a65 63740000   a type object..
 a030 53686172 65642043 7974686f 6e207479  Shared Cython ty
 a040 70652025 2e323030 73206861 73207468  pe %.200s has th
 a050 65207772 6f6e6720 73697a65 2c207472  e wrong size, tr
 a060 79207265 636f6d70 696c696e 67000000  y recompiling...
 a070 5f5f696e 745f5f20 72657475 726e6564  __int__ returned
 a080 206e6f6e 2d696e74 20287479 70652025   non-int (type %
 a090 2e323030 73292e20 20546865 20616269  .200s).  The abi
 a0a0 6c697479 20746f20 72657475 726e2061  lity to return a
 a0b0 6e20696e 7374616e 6365206f 66206120  n instance of a
 a0c0 73747269 63742073 7562636c 61737320  strict subclass
 a0d0 6f662069 6e742069 73206465 70726563  of int is deprec
 a0e0 61746564 2c20616e 64206d61 79206265  ated, and may be
 a0f0 2072656d 6f766564 20696e20 61206675   removed in a fu
 a100 74757265 20766572 73696f6e 206f6620  ture version of
 a110 50797468 6f6e2e00 5f5f696e 745f5f20  Python.. __int__
 a120 72657475 726e6564 206e6f6e 2d696e74  returned non-int
 a130 20287479 70652025 2e323030 73290000   (type %.200s)..
 a140 63616e27 7420636f 6e766572 74206e65  can't convert ne
 a150 67617469 76652076 616c7565 20746f20  gative value to
 a160 756e7369 676e6564 20696e74 00000000  unsigned int....
 a170 76616c75 6520746f 6f206c61 72676520  value too large
 a180 746f2063 6f6e7665 72742074 6f20756e  to convert to un
 a190 7369676e 65642069 6e740000 00000000  signed int......
 a1a0 496e7465 72707265 74657220 6368616e  Interpreter chan
 a1b0 67652064 65746563 74656420 2d207468  ge detected - th
 a1c0 6973206d 6f64756c 65206361 6e206f6e  is module can on
 a1d0 6c792062 65206c6f 61646564 20696e74  ly be loaded int
 a1e0 6f206f6e 6520696e 74657270 72657465  o one interprete
 a1f0 72207065 72207072 6f636573 732e0000  r per process...
 a200 756e626f 756e6420 6d657468 6f642025  unbound method %
 a210 2e323030 53282920 6e656564 7320616e  .200S() needs an
 a220 20617267 756d656e 74000000 00000000   argument.......
 a230 5f5f616e 6e6f7461 74696f6e 735f5f20  __annotations__
 a240 6d757374 20626520 73657420 746f2061  must be set to a
 a250 20646963 74206f62 6a656374 00000000   dict object....
 a260 5f5f6b77 64656661 756c7473 5f5f206d  __kwdefaults__ m
 a270 75737420 62652073 65742074 6f206120  ust be set to a
 a280 64696374 206f626a 65637400 00000000  dict object.....
 a290 6368616e 67657320 746f2063 7966756e  changes to cyfun
 a2a0 6374696f 6e2e5f5f 6b776465 6661756c  ction.__kwdefaul
 a2b0 74735f5f 2077696c 6c206e6f 74206375  ts__ will not cu
```

通过 objdump 分析 crypt_core.so 的 .rodata 段，结合初始化流程发现：

PyInit_crypt_core 函数会将地址 ac10~ad90 范围内的常量拷贝到运行时表。

加密算法为自定义的 SM4-like 分组密码，但具有以下特点：

轮数：24 轮（标准 SM4 为 32 轮）

分组大小：16 字节

填充方式：PKCS#7

使用自定义的 S 盒、FK 和 CK 常量

从 .rodata 中提取的常量如下：

S 盒：256 字节（地址 ac10 开始）

FK：4 个 32 位整数（地址 ad10 开始）

CK：24 个 32 位整数（地址 ad20 开始）

密钥：位于 ad80 开始的 32 字节中，其中前半部分 16 字节为实际密钥（小端序）：24f8d99215ed5ce765d85845cd5052be。

根据提取的常量和密钥，编写 Python 脚本实现 24 轮 SM4-like 解密（小端序），对 flag.txt 的密文进行解密，并去除 PKCS#7 填充，得到明文：

dart{f4b547fc-b3d0-44c3-bf21-8f3fb5ad3220}